Chapter 6

The µML State Model

Contents

6. The µML State Model

The *µML State Model* is a finite state machine model, which may be created by hand during the design phase or may be created automatically by model transformation. When sketched by hand, a State Model is drawn as a collection of states, connected by transitions labelled by events. When created by model transformation, a State Model is obtained from a *µML Task Model* by mapping clusters of tasks to states, representing the modes of a GUI for the application.

The purpose of the State Model is to represent the reactive behaviour of a system. Actions in the system are triggered by events, which are signals input by the user. The State Model defines the acceptable event protocol for the system, and also defines the state-dependent triggering of actions. In a State Model of a GUI, the events are button-clicks, the transitions are triggered processes, and the states are GUI screen views.

6.1 Elements of the State Model

The State Model consists of the following concepts:

- **State** – is a state of the system (or mode of its GUI) which is connected to other states by transitions. A state is owned by a machine and may optionally contain a nested machine.
- **Actor** – is any authorised user of the system, who may have permission to enter some states but not others. A state may optionally indicate its authorised actors.

The State Model also consists of the following connectives:

- **Transition** – connects a source state to a target state, labelled with an event. An initial transition has no source state; and a final transition has no target state.

The State Model also consists of the following features:

- **Property** – a kind of named feature that is owned by a transition.
- **Event** – a kind of property denoting a triggering event.
- **Action** – a kind of property denoting a triggered action.
- **Guard** – a kind of property denoting a conditional guard.

The State Model also has the following regions:

- **Machine** – is a kind of region containing a set of states and transitions. Each Machine is the root container for its own states and transitions. The top-level diagram is also a Machine.

6.2 Notation for the State Model

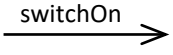
State: denotes a quiescent state of an entity, in between any activity that may cause its state to be changed. Drawn as a flattened oval with rounded ends.	
Transition: denotes the transfer of control from the source to the target state in response to an event. Labelled with the event and possibly a guard condition and a triggered action.	
Actor: denotes an authorised user, a stakeholder interacting in a particular role. Drawn as a stick figure with a label positioned below containing the name.	

Figure 6.1: Notation for the State Model

Figure 6.1 illustrates the notation used for concepts and connectives in the µML State Model. It adopts standard µML conventions in the concepts, connectives and features used. The icon for a state is a flattened oval. This has flat upper and lower edges, but ellipsoid ends, to distinguish the state icon from the µML process icon, which is drawn as a rectangle with rounded corners (see chapter 2). The icon for a transition is a labelled arrow with an open arrowhead.

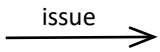
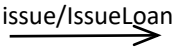
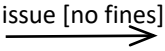
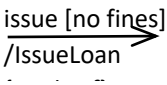
Simple Transition: denotes the processing of an event. Must always be labelled with the triggering event.	
Transition with Action: denotes the processing of an event, showing its triggered action. The action is preceded by a forward slash.	
Guarded Transition: denotes an event that is processed only when the guard is also true. The guard is enclosed in square brackets.	
Guarded Transition with Action: denotes an event that is processed only when the guard is also true, showing the triggered action. The features appear in this order.	

Figure 6.2: Different features of a transition

Figure 6.2 illustrates the different styles in which labelling features may be added to a transition. A transition must always have an event label. If the transition is guarded, then the guard condition appears next, enclosed in square brackets. If the transition also triggers an action, then the action appears last, prefixed by a forward slash. Every transition has its own unique labels. For convenience, multiple transitions from the same source to the same target may be elided as one transition labelled with a comma-separated set of labels. These are understood to denote multiple transitions.

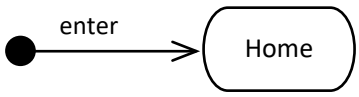
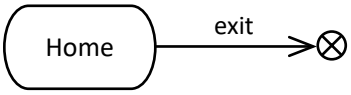
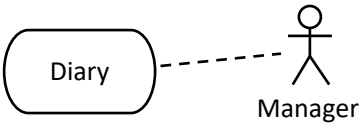
Initial Transition: denotes a transition entering the initial state. The empty source of the transition is drawn as a small, filled circle.	
Final Transition: denotes a transition exiting a final state. The empty target of the transition is drawn as a small circle containing a cross.	
Authorisation: denotes the granting of access to a given state by a named user. Drawn as a dashed line connecting the state to an actor.	

Figure 6.3: Initial and final transitions and authorisation.

Figure 6.3 illustrates the notation used for drawing initial and final transitions in a µML State Model. Logically, an initial transition has no source state, and a final transition has no target state. To make this clear in the notation, we use entry and exit icons, small circles, which are not states.

Figure 6.3 also shows how to indicate a state to which access is restricted to a given user of the system. This reuses the notation for an actor. The notation is optional and can be used in a system for multiple users with different levels of authorisation. A single actor icon can be placed in the top-left corner of the machine, to indicate that all states in the machine are authorised to this user. In a diagram with some authorised states, any unauthorised states are assumed to be open to any user.

6.3 Rules of the State Model

The State Model has the following syntactic and semantic rules:

- **Initial state** – a state machine must always have one initial state. This is the state targeted by an initial transition, which has no source state. This is indicated by drawing an arrow from the entry icon to the initial state (a definition rule).
- **Final state** – a state machine may have one or more final states. These are states from which transitions exit, but which have no target state. This is indicated by drawing an arrow from the final state to the exit icon (a definition rule).
- **Halting machine** – if any final transition is taken, the behaviour described by the state machine halts. This is true at any level of nesting (a semantic rule).
- **Perpetual machine** – if a state machine has no final transition, then the behaviour it describes will continue in perpetuity (a semantic rule).
- **Event triggers –** every transition must be triggered by an event. There are no triggerless, or instantaneous transitions (a definition rule).
- **Event processing** – when an event is processed, a state machine may take any transition exiting the current state labelled with this event. If there is no such transition, the event is ignored (a semantic rule).
- **Deterministic machine** – a state machine is deterministic if, for each of its states, the transitions exiting that state are labelled with mutually exclusive events (a semantic rule).
- **Nondeterministic machine** – a state machine is nondeterministic if, for at least one of its states, the transitions exiting that state are labelled with overlapping events (a semantic rule).
- **Enforced determinism** – if a state machine is nondeterministic, determinism can be enforced by adding mutually exclusive guards to the transitions with overlapping events (a semantic rule).
- **Nested machine** – a state machine containing nested state machines is a notational shorthand and may be flattened to reveal an equivalent flat state machine (a definition rule).
- **Superstate entry** – an entry transition to a superstate containing a nested state machine is equivalent to a transition to the initial state of that machine (a semantic rule).
- **Superstate exit** – an exit transition from a superstate containing a nested state machine is equivalent to replicated exit transitions from all states of that machine (a semantic rule).
- **Single-user authorisation** – a state machine with one actor icon placed in the machine boundary indicates that every state in that machine is authorised only to that user (a definition rule).
- **Multi-user authorisation** – a state machine in which specific users are authorised to enter certain states means that only the named user is authorised to enter that state, and any other state with no indicated authorisation is available to any user (a definition rule).
- **Implicit boundary** – the diagram is the implicit boundary of the State Model. It is equivalent to the top-level state machine (a completeness rule).

6.4 State Model Examples

Some examples are given below of State Models. The first example describes a Lending Library. The second example describes a Training Agency.

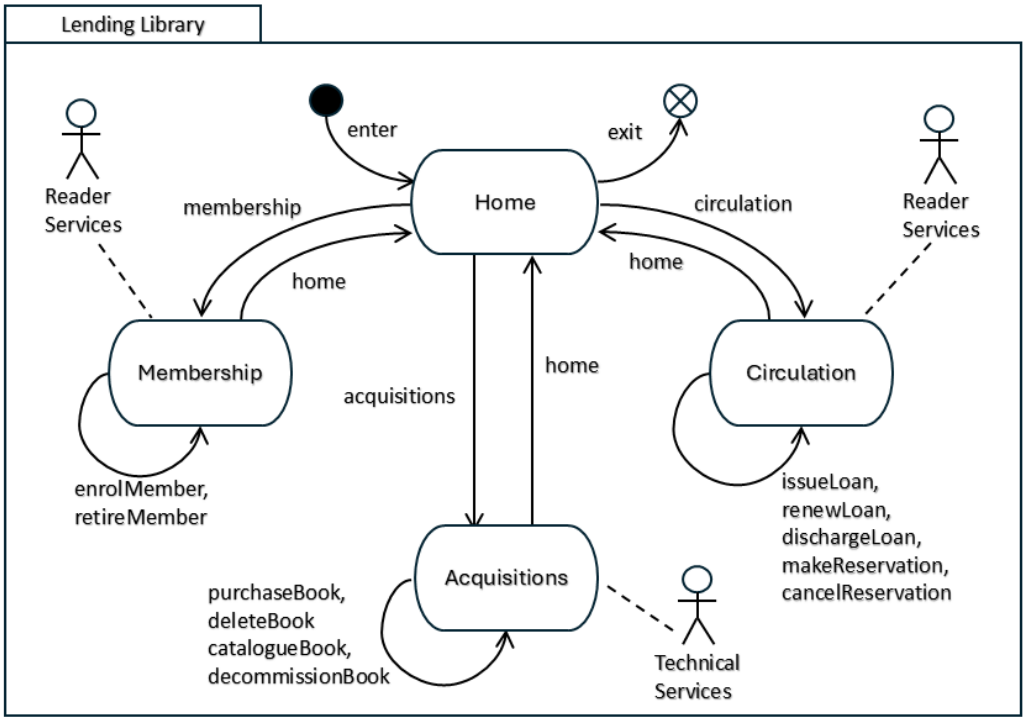


Figure 6.4: State Model for a Lending Library

Figure 6.4 illustrates simple State Model for a Lending Library, which issues loans of books to its members. The model contains four states and mentions two different users (one of them is repeated in the figure). The four states are:

- **Home** – denotes the home state, or home screen of the application. This is the initial state of the application, shown by the *enter* initial transition. It is also the final state of the application, shown by the *exit* transition.
- **Membership** – denotes the state, or screen, in which the library's membership may be updated. Transitions *enrolMember* and *retireMember* may be triggered in this state, and return to this state.
- **Acquisitions** – denotes the state, or screen, in which the library's collection of book titles and copies may be updated. Transitions *purchaseBook, catalogueBook* etc. may be triggered in this state, and return to this state.
- **Circulation** – denotes the state, or screen, in which the issuing and discharging of loans to members may be conducted. Transitions *issueLoan, dischargeLoan* etc. may be triggered in this state, and return to this state.

In this application, different states are authorised to different users:

- **Home** – is not restricted, so is authorised to all indicated users. These are: *Reader Services* and *Technical Services* (no other users are listed anywhere).
- **Membership** – is authorised to *Reader Services* (only).
- **Acquisitions** – is authorised to *Technical Services* (only).
- **Circulation** – is authorised to *Reader Services* (only).

It is possible to link an actor to more than one state; in this diagram, we simply repeated the *Reader Services* icon to avoid having long-range or crossing lines. All occurrences of the same named actor icon denote the same user. It is also possible to link a state with several actors, if more than one kind of user is authorised to enter that state.

Associating an actor with a state is equivalent to placing a guard condition on any transition entering that state. For example, we could write: *membership [user in {ReaderServices}]* to indicate that only a *Reader Services* user is allowed to enter the *Membership* state. The optional authorisation notation is a syntactic sugar for guards placed on transitions.

Note that a final state is one in which the user can exit the system, but they are not obliged to do so. The user may also choose to navigate from the *Home* state to other states. Note also that the only allowed exit is from the *Home* state. This means that a user may not exit the system from one of the other states (the implementation must prevent this).

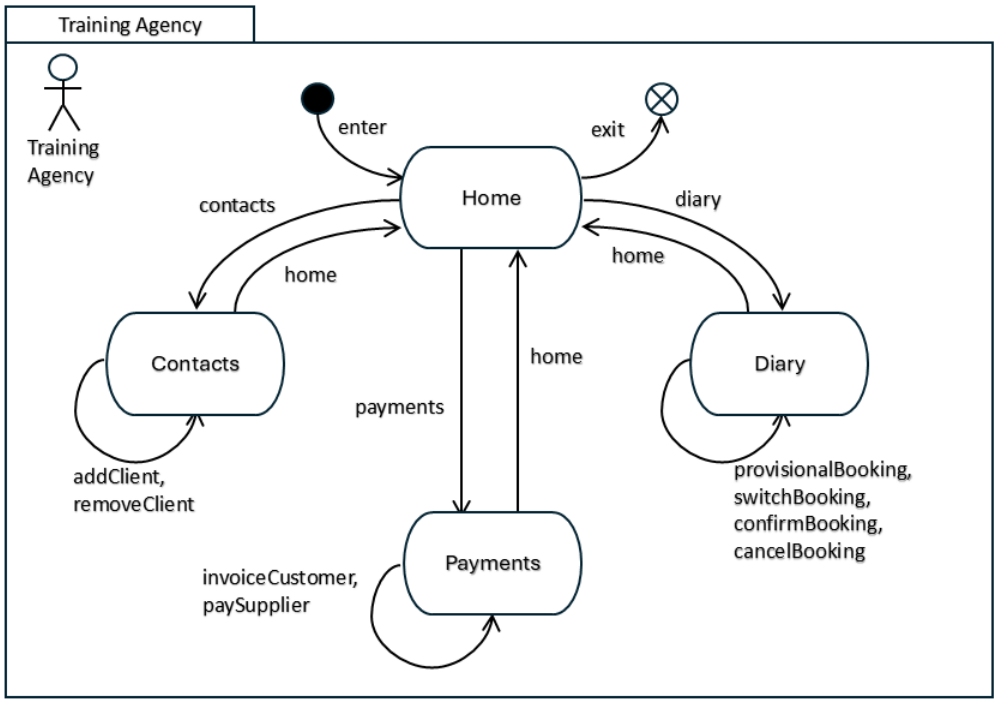


Figure 6.5: State Model for a Training Agency

Figure 6.5 illustrates a simpler State Model for a Training Agency, which arranges training sessions for trainees with trainers, booked into different venues. The model contains four states and only a single authorised user.

- **Home** – denotes the home state, or home screen of the application. This is the initial state of the application, shown by the *enter* initial transition. It is also the final state of the application, shown by the *exit* transition.
- **Contacts** – denotes the state, or screen, in which the Training Agency's clients may be updated. Transitions *addClient, removeClient* may be triggered in this state and return to this state.
- **Payments** – denotes the state, or screen, in which the Training Agency's customers may be billed and the suppliers may be paid. Transitions *invoiceCustomer*, *paySupplier* may be triggered in this state and return to this state.
- **Diary** – denotes the state, or screen, in which bookings may be added, switched, confirmed and deleted in the Training Agency's diary. Transitions *provisionalBooking, confirmBooking* etc. may be triggered in this state and return to this state.

In this application, all states of the machine are authorised to a single user, *Training Agency*. This is indicated using the shorthand of placing an actor icon within the state machine boundary. This is deemed equivalent to associating the actor with every state in the machine.

6.5 State Metamodel

The elements of the µML State Model may be defined in a metamodel. In figure 6.6, we use a metamodel written in the *ReMoDeL* metamodel definition language [1]. This extends and redefines concepts defined in the ReMoDeL metamodel of the µML Core Model, given in Chapter 2.

```
# The State Model from mML (micro Modelling Language).  This consists of
# a Machine containing States and Transitions.  States are named and may
# represent data states or screen states in a GUI.  Transitions run from
# the source State to the target State, and are labelled with an Event and
# optionally a Guard and an Action.  The top-level Machine also lists the
# authorised Actors.  States may be composite, containing nested Machines
# that expose more detailed behaviour.  A hierarchical Machine is a short-
# hand, equivalent to a flat Machine with more Transitions.

metamodel StateModel inherit CoreModel {

# Actor is a kind of Concept denoting an authorised user of the system.
# Actors may be referenced by a State, but are owned by the top level
# Machine.

   concept Actor inherit Concept {
      reference owner : Machine
   }

# Property is a kind of Feature denoting a named property of a transition.
# Such properties include Event, Guard and Action.  A Property is owned
# by a Transition.

  concept Property inherit Feature {
    reference owner : Transition
  }

# Event is a kind of Property denoting a triggering event.  The
# Event has a name, which is the triggering signal.

  concept Event inherit Property {
  }

# Action is a kind of Property denoting a triggered procedure.
# An Action is named and this refers to the name of the triggered
# procedure.

   concept Action inherit Property {
   }

# Guard is a kind of Property denoting a conditional guard.  A Guard
# can be named, but also has a conditional guard expression.

   concept Guard inherit Property {
      attribute guard : String
   }

# State is a kind of Concept denoting a named state.  A State may admit one or
# more Users.  A State has possibly many incoming and outgoing Transitions.
# A State may be an initial, or final State if it is connected to an initial,
# or final transition.  A State may be composite if it has a nested Machine.
# A State is owned by a Machine.

   concept State inherit Concept {
      reference owner : Machine
      reference actors : Actor[]
```

```
      component machine : Machine

      operation outgoing : Transition[] {
         owner.transitions.select(trans : Transition |
            trans.source = self)
      }
      operation incoming : Transition[] {
         owner.transitions.select(trans : Transition |
            trans.target = self)
      }
      operation isComposite : Boolean {
         machine /= null
      }
      operation isInitial : Boolean {
         owner.transitions.exists(trans : Transition |
            trans.isInitial and trans.target = self)
      }
      operation isFinal : Boolean {
         owner.transitions.exists(trans : Transition |
            trans.isFinal and trans.source = self)
      }
   }

# A Transition is a kind of Connective denoting a Transition.  A
# Transition is named, connects a source State and target State,
# has a triggering event, an optional Guard and contains a triggered
# Action.

   concept Transition inherit Connective {
      reference owner : Machine
      reference source : State
      reference target : State
      component event : Event
      component guard : Guard
      component action : Action
      operation isInitial : Boolean { source = null }
      operation isFinal : Boolean { target = null }
   }

# A Machine is a kind of Region enclosing a collection of States and
# Transitions.  The Machine owns its States and Transitions.  If the
# Machine is nested inside a composite State, then its owner is that
# State.  The top-level Machine also lists the known Actors that may
# be authorised to enter some of its States.

   concept Machine inherit Region {
      reference owner : State
      component actors : Actor[]
      component states : State[]
      component transitions : Transition[]
      operation isNested : Boolean {
         owner /= null
      }
   }

}
```

Figure 6.6: Metamodel of the State Model in the ReMoDeL language.

This provides the necessary elements of the State Model via a number of intermediate abstractions in the metamodel hierarchy:

- **Actor** – is a kind of core Concept denoting a user, and is owned by a Machine;
- **State** – is a kind of core Concept denoting a state, and is owned by a Machine. A State may refer to the Actors authorised to enter it. A state may contain a nested Machine;

- **Transition** – is a kind of core Connective, denoting a transition from one state to another, and is owned by a Machine, and is labelled with one or more Properties;
- **Property** – is a kind of core Feature, an abstract property owned by a Transition.
- **Event** – is a kind of Property denoting a triggering event;
- **Action** – is a kind of Property denoting a triggered action;
- **Guard** – is a kind of Property denoting a Boolean guard – the condition is expressed as a textual expression;
- **Machine** – is a kind of core Region owning the states and transitions of that machine. The top-level Machine lists all authorised Actors. A nested Machine will have a State owner.

Only the top-level machine, the diagram containing the whole State Model, contains the complete list of authorised users, and other states may refer to subsets of these users. (The reason for this is to ensure that serialised model instances list the Actors in full at the top of the model file, and refer to them by ID later in the model file).

6.6 Model Transformations

The µML State Model is a source or target in a number of model transformations. Some of these are to generate the State Model from other models, or to combine a State Model with other State Models, or to flatten a hierarchical State Model to yield a flat State Model.

- **TaskToStateModel** – is a mapping transformation that takes a µML Task Model as the source, and from this constructs a µML State Model. The result is a finite state machine of the user interface for the business application represented by the original Task Model. Composite tasks are mapped to states, and atomic tasks are mapped to transitions in those states. Additional transitions are created to map between a home state and the other derived states.
- **StateToLoginModel** – is a mapping transformation that takes a µML State Model as the source, and from this constructs another µML State Model. The source model is a state machine representing the business logic of an application (such as in figures 6.4 and 6.5). The target model is a machine representing a login authentication process, in which the source machine is nested inside the logged-in state. This adds an authentication layer to any state machine.
- **EmbedStateModel** – is a merging transformation that takes two existing µML State Models as the sources, and from this constructs a hierarchical target µML State Model, in which the second source model is nested inside one of the states of the first source model. The name of the nested machine must correspond to the name of the intended composite state. The nested machine is authorised to any user authorised to enter the composite state.
- **MergeStateModels** – is a mapping transformation that takes two existing µML State Models as the sources, and from this constructs a merged target µML State Model, which contains the Cartesian product of the states of the source models. This transformation assumes that the merged machines capture orthogonal kinds of behaviour. State names are synthesised from unique pairs of source state names. Transitions are replicated. Authorisation is handled by computing the most restricted authorisation for the product state.
- **ToFlatStateModel** – is a mapping transformation that takes an existing hierarchical µML State Model as the source, and from this constructs a flat µML State Model as the target. The result expands out any composite state containing a nested state machine, such that entry transitions to the composite state target the initial state of the nested machine, and exit transitions from the composite state are replicated as transitions from every state of the nested machine.

These transformations are too large to display in full here.

6.7 Differences from UML

The µML State Model is somewhat different from, and significantly simpler than the UML State Machine Diagram. There are many theoretical precursors in classic finite state machine theory [2, 3], in extended finite state machines [4, 5] and in labelled transition systems [6, 7]. The µML State Model is influenced by Harel Statecharts [8] and to a lesser extent by the UML State Machine Diagram [9].

In the early *Mealy Machine* [2], the machine reacts to inputs in a given state, triggering the transition from that state labelled with that input, and generates a corresponding output depending on the transition fired. In a *Moore Machine* [3] this is simplified such that the outputs correspond to the target state reached. Mealy Machines may be used as *recognisers* (parsing the input sequence), as *generators* (generating all legal output sequences) and *transducers* (converting an input sequence to an output sequence).

Formal methods, such as process algebra [6] make use of connected graphs of states, sometimes known as Kripke structures [7] to reason about the future of a system. These mathematical approaches to modal and temporal logic also consider nondeterminism, when the transition to be fired is not unique. State machines used in software engineering are typically Extended Finite State Machines (EFSMs), which add small extensions to a simple FSM [4]. The simplest extension is memory, one or more variables that can be read or set by actions on the transitions. Eilenberg's X-machine could perform Turing-complete computation by manipulating memory [4]. Laycock's Stream X-machines [5] ensured that these machines were controllable, with every transition also necessarily triggered by an input.

Hierarchical state machines were introduced by Harel [8] and were subsequently adopted by UML [9]. The µML State Model adopts Harel's synchronous semantics (events are processed to completion much faster than the next event can be raised) and Harel's transition conflict resolution rule (if two transitions in a nested and outer machine could fire, the outer transition is always taken). We don't expect transition conflicts to occur across state hierarchies, because the machines denote different levels of event processing.

Our notation differs from that used by UML, which uses the same notation for *Actions* (in the UML Activity Diagram) and *States* (in the UML State Machine Diagram). This notational confusion is only superficial, but the problem is then exacerbated in UML, which allows a State to denote processing, and also allows triggerless exit transitions when that processing has finished. This possibility makes the two UML models indistinguishable. So, we insist on a reactive semantics, similar to Mealy, in which every transition must be triggered by an event. All processing is performed by triggered actions on the transitions, rather than by entering or exiting states.

Nested machines in the µML State Model are also simpler than in UML. We only allow a state to contain a single nested machine, rather than have multiple partitions containing orthogonal, concurrent state machines. We take the view that state hierarchy is formally just a shorthand for the expanded flat machine. We expect to rely on model transformations to compose orthogonal machines, or to flatten nested state machines when this is needed.

The usage of the µML State Model described in this chapter relates strongly to the overall behaviour of a system, rather than the detailed behaviour of one system component. We are focusing here more on the behaviour of the GUI of the system, than on the detailed behaviour of its subsystems and components. This does not rule out the possibility of a later adaptation of the notation to represent the reactive behaviour of components.

6.8 References

[1] AJH Simons. ReMoDeL Explained (rev 2.2): an introduction to ReMoDeL by example. *Technical Report, 25 January*, University of Sheffield (2023). https://staffwww.dcs.shef.ac.uk/people/A.Simons/remodel/current/ReMoDeL_Explained_25Jan26.pdf

[2] GH Mealy. A method for synthesizing sequential circuits, *Bell System Technical Journal* (1955), 1045–1079.

[3] EF Moore. Gedanken-experiments on sequential machines, *Automata Studies, Annals of Mathematical Studies*, **34**, Princeton University Press (1956), 129–153.

[4] S. Eilenberg. *Automata, Languages and Machines, Vol. A*. (London: Academic Press) (1974).

[5] G Laycock. *The Theory and Practice of Specification Based Software Testing. PhD Thesis,* University of Sheffield (1993).

[6] JA Bergstra, A Ponse and SA Smolka. *A Handbook of Process Algebra*, Elsevier (2001).

[7] S Kripke. Semantical considerations on modal logic, *Acta Philosophica Fennica*, **16** (1963), 83-94.

[8] D Harel. Statecharts: a visual formalism for complex systems, *Science of Computer Programming*, **8** (3), June (1987), 231-274.

[9] G Booch, J Rumbaugh, I Jacobson. *The Unified Modeling Language User Guide*, Addison-Wesley (1999).